

# Docker Dockerfile CheatSheet - Modul 347

---

## Inhaltsverzeichnis

- Docker Dockerfile CheatSheet - Modul 347
  -  Inhaltsverzeichnis
  -  Grundlegende Dockerfile-Struktur
  -  Dockerfile-Befehle im Detail
    - FROM
    - LABEL
    - ARG vs ENV
      - ARG (nur Build-Zeit)
      - ENV (Build + Laufzeit)
    - WORKDIR
    - COPY vs ADD
      - COPY (Empfohlen)
      - ADD (Erweitert)
    - RUN
    - EXPOSE
    - CMD vs ENTRYPOINT
      - CMD (überschreibbar)
      - ENTRYPOINT (fest)
      - Kombination ENTRYPOINT + CMD
  -  Python-Anwendungen
    - Einfache Python-App
    - Flask-Anwendung
    - FastAPI-Anwendung
  -  Build & Run Befehle
    - Image bauen
    - Container starten
  -  Debugging & Inspection
    - Container untersuchen
    - Image untersuchen
  -  Wichtige Hinweise
    - Shell-Form vs Exec-Form
    - Sicherheit
    - Best Practices
  -  Typische Aufgaben-Patterns
    - 1. EXPOSE-Test
    - 2. LABEL-Test
    - 3. ARG/ENV-Test

## 🔑 Grundlegende Dockerfile-Struktur

```
FROM <base-image>
LABEL <key>=<value>
ARG <variable>=<default>
ENV <key>=<value>
WORKDIR <path>
COPY <src> <dest>
ADD <src> <dest>
RUN <command>
EXPOSE <port>
ENTRYPOINT ["executable", "param1"]
CMD ["param1", "param2"]
```

## 📋 Dockerfile-Befehle im Detail

### FROM

```
FROM debian
FROM python:3.11
FROM python:3.11-slim
FROM nginx
```

- **Zweck:** Definiert das Basis-Image
- **Position:** Immer der erste Befehl (außer ARG vor FROM)

### LABEL

```
LABEL version="1.0"
LABEL ch.bbzw.m347.label="Docker Kurs"
LABEL maintainer="name@example.com"
```

- **Zweck:** Metadaten zum Image hinzufügen
- **Best Practice:** Umgedrehte Domainnamen als Präfix verwenden
- **Anzeigen:** `docker image inspect -f "{{.Config.Labels}}" <image>`

### ARG vs ENV

#### ARG (nur Build-Zeit)

```
ARG install_software=python3
RUN apt update && apt install -y $install_software
```

- **Verwendung:** `docker build --build-arg install_software="python3 python3-pip"`
- **Sichtbar:** Nur während Build-Prozess

#### ENV (Build + Laufzeit)

```
ENV FLASK_APP=app.py
ENV FLASK_RUN_PORT=80
ENV PROGRAM=python3
```

- **Überschreiben:** `docker run -e PROGRAM="python3 -OO" <image>`
- **Sichtbar:** Build-Zeit und Container-Laufzeit

## WORKDIR

```
WORKDIR /app
WORKDIR /projekt
```

- **Zweck:** Arbeitsverzeichnis für folgende Befehle setzen
- **Erstellt:** Verzeichnis automatisch, falls nicht vorhanden

## COPY vs ADD

### COPY (Empfohlen)

```
COPY app.py .
COPY src/ /app/
COPY requirements.txt /app/
```

- **Zweck:** Lokale Dateien/Ordner in Container kopieren
- **Einfach:** Nur Kopieren, keine zusätzlichen Features

### ADD (Erweitert)

```
ADD Webseite.tar.bz2 /usr/share/nginx/html
ADD https://example.com/file.txt /app/
```

- **Zusatzfeatures:** Automatisches Entpacken von Archives
- **Remote:** URLs herunterladen möglich

## RUN

```
RUN apt update && apt install -y python3
RUN pip install -r requirements.txt
RUN chmod +x /app/start.sh
```

- **Zweck:** Befehle während Build-Prozess ausführen
- **Best Practice:** Mehrere Befehle mit **&&** verknüpfen

## EXPOSE

```
EXPOSE 80
EXPOSE 443/tcp
EXPOSE 8888
```

- **Zweck:** Dokumentiert verwendete Ports
- **Wichtig:** Startet KEINE automatische Portweiterleitung!
- **Automatisches Mapping:** `docker run -P <image>` (großes P)

## CMD vs ENTRYPOINT

### CMD (überschreibbar)

```
CMD ["python3", "app.py"]
CMD ["bash"]
```

- **Überschreiben:** `docker run <image> bash`
- **Standard-Parameter** für Container

### ENTRYPOINT (fest)

```
ENTRYPOINT ["python3", "app.py"]
ENTRYPOINT ["ping"]
```

- **Nicht überschreibbar** (außer mit `--entrypoint`)
- **Hauptbefehl** des Containers

### Kombination ENTRYPOINT + CMD

```
ENTRYPOINT ["python3"]
CMD ["app.py"]
```

- **Flexibel:** `docker run <image> --version`
- **Standard:** `docker run <image>` führt `python3 app.py` aus

## Python-Anwendungen

### Einfache Python-App

```
FROM python:3.11
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "app.py"]
```

### Flask-Anwendung

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 80
ENV FLASK_APP=app.py
ENV FLASK_RUN_PORT=80
CMD ["flask", "run", "--host=0.0.0.0"]
```

## FastAPI-Anwendung

```
FROM python:3.11
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

## Build & Run Befehle

### Image bauen

```
docker build -t <name> .
docker build -t <name>:<tag> .
docker build --build-arg <arg>=<value> -t <name> .
```

### Container starten

```
# Standard
docker run <image>

# Interaktiv
docker run -it <image>

# Hintergrund
docker run -d <image>

# Port-Mapping
docker run -p 8080:80 <image>

# Automatisches Port-Mapping
docker run -P <image>

# Umgebungsvariablen
docker run -e VAR=value <image>

# Temporär (auto-remove)
docker run --rm <image>

# CMD überschreiben
docker run <image> bash

# ENTRYPOINT überschreiben
docker run --entrypoint=bash <image>
```

## Debugging & Inspection

### Container untersuchen

```
# Laufende Container
docker container ls

# Container-Details
docker container inspect <container>

# In Container einsteigen
docker container exec -it <container> bash

# Logs anzeigen
docker container logs <container>
```

### Image untersuchen

```
# Images auflisten
docker image ls

# Image-Details
docker image inspect <image>

# Labels anzeigen
docker image inspect -f "{{.Config.Labels}}" <image>

# Spezifisches Label
docker image inspect -f "{{.Config.Labels.version}}" <image>

# Image-Schichten
docker image history <image>
```

## Wichtige Hinweise

### Shell-Form vs Exec-Form

```
# Shell-Form (Variablen werden ausgewertet)
CMD $PROGRAM main.py
RUN echo $PATH

# Exec-Form (Empfohlen, direkte Ausführung)
CMD ["python3", "main.py"]
ENTRYPOINT ["python3"]
```

### Sicherheit

- **Niemals** Passwörter oder Tokens in ARG/ENV
- Beide sind in `docker image history` sichtbar!
- Für Secrets: Docker Secrets oder externe Systeme verwenden

### Best Practices

1. `.dockerignore` verwenden für unnötige Dateien
2. Multi-Stage Builds für kleinere Images
3. Spezifische Tags statt `latest`
4. Wenige RUN-Schichten (mit `&&` verknüpfen)
5. COPY vor ADD bevorzugen
6. Exec-Form für CMD/ENTRYPOINT bevorzugen

## Typische Aufgaben-Patterns

### 1. EXPOSE-Test

```
FROM debian
EXPOSE 80/tcp
EXPOSE 443/tcp
CMD ["bash"]
```

```
docker build -t expose-test .
docker run -d expose-test sleep 5000 # Keine Port-Weiterleitung
docker run -dP expose-test sleep 5000 # Automatische Port-Weiterleitung
```

### 2. LABEL-Test

```
FROM ubuntu
LABEL version="1.0"
LABEL ch.bbzw.m347.label="Docker Kurs"
```

```
docker build -t label-test .
docker image inspect -f "{{.Config.Labels}}" label-test
```

### 3. ARG/ENV-Test

```
FROM debian
ARG install_software=python3
RUN apt update && apt install -y $install_software
ENV PROGRAM=python3
CMD ["sh", "-c", "$PROGRAM --version"]
```

```
docker build --build-arg install_software="python3 python3-pip" -t arg-env-test .
docker run -e PROGRAM="python3 -OO" arg-env-test
```

---

*Erstellt für Modul 347 - Docker Prüfungsvorbereitung*